Sovereign AI Memory: Persistent, Private, and Portable Agent State on Zoo Network

Encrypted Memory Capsules with Threshold Access Control

Version 2026.04

Zoo Labs Foundation

2026

Abstract

Contemporary AI agents suffer from persistent amnesia. Each session begins tabula rasa; memory, when it exists, lives in provider-controlled vector databases that the agent cannot own, move, or protect. We argue that AI memory is not a retrieval problem—it is a *sovereignty* problem. This paper introduces **Memory Capsules**, encrypted append-only vaults that give agents persistent, private, and portable state on Zoo Network. A Memory Capsule binds to a decentralized identity (DID) on I-Chain, stores content-addressed entries in an encrypted SQLite vault sealed with K-Chain-managed keys, synchronizes across agent instances via operation-based CRDTs, and enforces fine-grained access through capability tokens and T-Chain threshold reveal. We formalize four memory types—episodic, semantic, procedural, and preference—and show how each maps to distinct encryption and garbage-collection policies. For sensitive memories, we demonstrate FHE-encrypted reasoning via CKKS on A-Chain: an agent can query its own memory without decrypting it, and third parties can verify reasoning traces without observing content. Cross-agent knowledge transfer is achieved through selective disclosure using zero-knowledge proofs, enabling collaborative intelligence without surrendering private state. We benchmark Memory Capsule operations on Zoo Network, measuring append latency (1.2 ms), CRDT merge throughput (14k ops/s), and FHE query overhead (340 ms per encrypted recall). The architecture positions AI memory as a first-class on-chain asset: ownable, transferable, and economically valuable.

Keywords: AI memory, sovereign state, encrypted vault, CRDT, FHE, capability-based access, decentralized identity

Contents

1 Introduction

1.1 The Amnesia Problem

Every major AI system today—GPT-4, Claude, Gemini—operates without durable memory. Context windows provide the illusion of continuity, but the moment a session ends, accumulated knowledge vanishes. Provider-side workarounds (conversation history, retrieval-augmented generation, user preference stores) do not solve the fundamental problem; they merely move it behind an API boundary controlled by the provider.

Three failures characterize the current state:

1. **No Persistence Across Sessions.** An agent that spent hours learning a user's codebase starts from scratch the next morning. Knowledge invested is knowledge lost.
2. **No Privacy From Providers.** When memory exists (e.g., OpenAI's memory feature), the provider stores it in plaintext, indexes it for product improvement, and retains the right to inspect, modify, or delete it.
3. **No Portability Across Models.** A user who switches from Claude to GPT cannot bring accumulated context. Memory is locked to a single vendor, creating switching costs that have nothing to do with model quality.

These are not engineering oversights. They are structural consequences of a centralized architecture where the model provider controls the memory layer.

1.2 Memory as Sovereignty

We reframe AI memory as a sovereignty problem with three requirements:

Definition 1 (Sovereign AI Memory)**.** A memory system is *sovereign* if and only if:

1. The agent (or its owner) holds the encryption keys—not the inference provider.
2. The memory persists independently of any single model, provider, or session.
3. The owner can grant, revoke, and scope access without provider cooperation.

No existing system satisfies all three conditions. Vector databases (Pinecone, Weaviate, Chroma) satisfy none: the operator holds keys, persistence depends on the operator's infrastructure, and access control is coarse-grained ACLs.

1.3 Contribution

This paper presents the Memory Capsule architecture on Zoo Network. Our contributions:

1. **Memory Capsule Specification.** A formal model for encrypted, content-addressed, append-only agent memory with DID-bound ownership (Section 3).
2. **Capability-Based Access Control.** Fine-grained, delegable, revocable access tokens authenticated via I-Chain DIDs and enforced by T-Chain threshold decryption (Section 4).
3. **Memory Type Taxonomy.** Four memory types (episodic, semantic, procedural, preference) with distinct storage, encryption, and retention policies (Section 5).

4. **FHE-Encrypted Memory Reasoning.** CKKS-based homomorphic computation over encrypted memory on A-Chain, enabling agents to reason over state they cannot decrypt (Section 6).

5. **Cross-Agent Knowledge Transfer.** Zero-knowledge selective disclosure for sharing knowledge between agents without revealing private state (Section 7).

6. **Benchmarks.** End-to-end measurements on Zoo Network testnet (Section 10).

1.4 Paper Organization

Section 2 motivates sovereign memory. Section 3 defines the capsule architecture. Section 4 covers access control. Section 5 formalizes memory types. Section 6 presents FHE-encrypted reasoning. Section 7 describes cross-agent transfer. Section 8 analyzes memory economics. Section 9 details implementation on Zoo's multi-chain architecture. Section 10 presents benchmarks. Section 11 surveys related work. Section 12 concludes.

2 Why Sovereign AI Memory Matters

2.1 Persistence Across Sessions

Long-running agent tasks—multi-day research projects, ongoing code refactoring, iterative design work—require memory that outlives individual inference calls. Current systems approximate persistence through conversation logs, but these degrade rapidly: a 200k-token conversation history is neither searchable nor structured, and exceeding the context window forces lossy summarization.

A sovereign memory system treats session boundaries as irrelevant. The capsule persists on-chain; the agent attaches to it at session start and detaches at session end. No information is lost.

2.2 Privacy From Providers

When an agent processes medical records, legal documents, or proprietary code, the memory it forms is as sensitive as the source material. Provider-controlled memory creates an untenable situation: the user must trust the provider not to inspect, sell, or leak derived knowledge.

Sovereign memory eliminates this trust requirement. The capsule is encrypted with keys the provider never sees. The provider runs inference; the agent manages its own state.

2.3 Portability Across Models

Model capability evolves rapidly. Users should be free to migrate from one model to another without losing accumulated context. Sovereign memory, stored in a model-agnostic format and encrypted with user-controlled keys, makes migration a key-management operation rather than a data-migration project.

2.4 Agent Autonomy

Autonomous agents—those operating with delegated authority over wallets, APIs, and infrastructure—need memory they control. An Agent NFT [5] that cannot remember its own transaction history, learned preferences, or accumulated expertise is not autonomous in any meaningful sense. Memory Capsules provide the state layer that Agent NFTs require.

3 Memory Capsule Architecture

3.1 Overview

A Memory Capsule is a tuple $\mathcal{C} = (did, log, vault, caps, meta)$ where:

- $did \in \mathcal{D}$ is the owner's decentralized identifier on I-Chain.
- *log* is a content-addressed append-only operation log.
- *vault* is an encrypted SQLite database storing materialized state.
- *caps* is the set of outstanding capability tokens.
- *meta* is an on-chain metadata record (creation time, size, type distribution, access statistics).

3.2 Content-Addressed Append-Only Log

Every mutation to a capsule is recorded as an operation in an append-only log. Operations are content-addressed using BLAKE3:

Definition 2 (Memory Operation)**.** An operation $o = (t, type, payload, h_{prev})$ consists of a timestamp t, operation type $type \in \{\texttt{write}, \texttt{update}, \texttt{delete}, \texttt{merge}\}$, an encrypted payload, and the hash of the previous operation $h_{prev} = \text{BLAKE3}(o_{prev})$.

This structure provides:

1. **Tamper Evidence.** Any modification to a historical entry breaks the hash chain.
2. **Causal Ordering.** The h_{prev} link establishes a total order within a single writer and a partial order across concurrent writers.
3. **Efficient Sync.** Two replicas can determine their divergence point by comparing head hashes.

The log is stored on Zoo Network's storage layer, with the head hash anchored on I-Chain for tamper-evident auditability.

3.3 Encrypted SQLite Vault

The materialized state—the current snapshot of all live memory entries—is stored in a SQLite database encrypted with SQLCipher (AES-256-CBC with HMAC-SHA512 page authentication). The database schema is fixed:

```
CREATE TABLE memories (
    id          BLOB PRIMARY KEY,  -- BLAKE3(content)
    type        INTEGER NOT NULL,  -- 0=episodic,1=semantic,
                                   -- 2=procedural,3=preference
    content     BLOB NOT NULL,     -- encrypted entry
    embedding   BLOB,              -- encrypted vector
    created_at  INTEGER NOT NULL,  -- unix timestamp
    accessed_at INTEGER NOT NULL,
    access_count INTEGER DEFAULT 0,
    ttl         INTEGER,           -- seconds, NULL=permanent
```

```
    tags        TEXT                -- JSON array, encrypted
);

CREATE TABLE operations (
    seq         INTEGER PRIMARY KEY AUTOINCREMENT,
    hash        BLOB NOT NULL UNIQUE,
    prev_hash   BLOB NOT NULL,
    op_type     INTEGER NOT NULL,
    payload     BLOB NOT NULL,
    timestamp   INTEGER NOT NULL
);
```

The vault key is derived from the owner's K-Chain key hierarchy:

$$k_{vault} = \text{HKDF-SHA256}(k_{master}, \text{“memory-capsule”} \| did) \tag{1}$$

where k_{master} is the owner's root key managed by K-Chain validators.

3.4 CRDT Synchronization

Agents may run across multiple instances (e.g., a desktop session and a mobile session operating concurrently). Memory Capsules synchronize via operation-based CRDTs:

Definition 3 (Memory CRDT)**.** The memory state is modeled as a grow-only set G of operations with a merge function $\sqcup$ defined as set union, and a *materialize* function $M(G)$ that replays operations in causal order to produce the current vault state.

Conflict resolution follows last-writer-wins (LWW) semantics for updates to the same key, with ties broken by DID-derived priority. Deletes are soft (tombstone markers) until garbage collection compacts them.

The synchronization protocol:

Algorithm 1 CRDT Sync Between Capsule Replicas

Require: Replicas R_A, R_B with head hashes h_A, h_B

R_A sends h_A to R_B
if $h_A = h_B$ **then**
  Replicas are synchronized; no action needed
else
  R_B computes $\Delta = log_B \setminus ancestors(h_A)$
  R_B sends Δ to R_A
  R_A validates each $o \in \Delta$ (signature, capability check)
  R_A merges: $G_A \leftarrow G_A \sqcup \Delta$
  R_A re-materializes vault: $vault_A \leftarrow M(G_A)$
  R_A anchors new head hash to I-Chain
end if

3.5 On-Chain Metadata

The full vault is too large for on-chain storage. Only metadata is anchored on I-Chain:

```
struct CapsuleAnchor {
    did:        bytes32,    // owner DID
    head_hash:  bytes32,    // current log head
    entry_count: uint64,
    total_bytes: uint64,
    type_dist:  [4]uint64, // count per memory type
    updated_at: uint64
}
```

This allows on-chain verification of capsule integrity without storing content on-chain.

4 Memory Access Control

4.1 Capability-Based Model

Access to a Memory Capsule is governed by *capability tokens*—unforgeable, delegable, scopeable authorization objects. Unlike ACLs (which attach permissions to identities), capabilities attach permissions to tokens that can be passed between agents.

Definition 4 (Memory Capability)**.** A capability $c = (\mathit{issuer}, \mathit{holder}, \mathit{scope}, \mathit{expiry}, \sigma)$ where:

- *issuer* is the DID of the capsule owner.
- *holder* is the DID of the authorized agent (or $\perp$ for bearer tokens).
- *scope* $\subseteq \{\texttt{read}, \texttt{write}, \texttt{delete}\} \times \mathcal{T} \times \mathcal{F}$ specifies allowed operations, memory types $\mathcal{T}$, and optional tag filters $\mathcal{F}$.
- *expiry* is a UNIX timestamp after which the capability is void.
- σ is the issuer's signature over the above fields.

4.2 DID Authentication

All capability checks begin with DID authentication on I-Chain. The requesting agent presents its DID and a fresh challenge-response signature. I-Chain resolves the DID document, extracts the verification key, and validates the signature.

4.3 Threshold Reveal for Sensitive Memories

Certain memories are too sensitive for any single key to unlock. These are encrypted under a threshold scheme managed by T-Chain:

1. The memory entry is encrypted with a symmetric key k_e.
2. k_e is split into n shares via Shamir's Secret Sharing, distributed to T-Chain validators.
3. Decryption requires t-of-n validators to contribute their shares (default: $t = \lceil 2n/3 \rceil$).
4. The requesting agent must present a valid capability token *and* achieve quorum.

This prevents both the agent owner (who cannot unilaterally decrypt) and any subset of validators (who lack the capability token) from accessing the memory alone.

4.4 Delegation and Revocation

Capabilities support bounded delegation:

- **Attenuation.** A holder can create a derived capability with equal or narrower scope. A read-write capability can be attenuated to read-only; a full-type capability can be narrowed to episodic-only.
- **Depth Limits.** Each capability carries a delegation depth counter. The issuer sets the maximum depth; each delegation decrements it.
- **Revocation.** The issuer publishes a revocation record to I-Chain. Validators check the revocation list before honoring capabilities. Revocation propagates within one I-Chain block confirmation ($\sim$2 seconds).

5 Memory Types

We formalize four memory types, each with distinct storage semantics, encryption policies, and retention behavior.

5.1 Episodic Memory

Definition 5 (Episodic Memory)**.** Episodic entries record specific events: conversations, transactions, observations, errors. Each entry is a timestamped record $e = (t, \mathit{context}, \mathit{content}, \mathit{salience})$ where salience $\in [0, 1]$ determines retention priority.

Properties:

- **Encryption.** AES-256-GCM per entry. Key derived from vault key and entry ID.
- **Retention.** Salience-weighted decay. Entries with $\mathit{salience} < \theta_{gc}$ (default 0.1) are eligible for garbage collection after their TTL expires.
- **Volume.** Highest volume type. Expected to dominate storage costs.
- **Access Pattern.** Temporal locality—recent episodes accessed most frequently.

5.2 Semantic Memory

Definition 6 (Semantic Memory)**.** Semantic entries encode learned facts, relationships, and world knowledge: "Project X uses PostgreSQL 16," "User prefers functional style." Each entry is a triple $s = (\mathit{subject}, \mathit{predicate}, \mathit{object})$ with an associated confidence score $\gamma \in [0, 1]$.

Properties:

- **Encryption.** AES-256-GCM. Embeddings are separately encrypted for vector search.
- **Retention.** Confidence-weighted. Entries reinforced by repeated observation increase γ; contradicted entries decrease. At $\gamma < 0.05$, eligible for removal.
- **Deduplication.** New entries are compared against existing semantic memory. If an entry with cosine similarity > 0.95 exists, the existing entry is updated rather than duplicated.

5.3 Procedural Memory

Definition 7 (Procedural Memory)**.** Procedural entries encode learned skills and workflows: tool-use sequences, API interaction patterns, multi-step reasoning chains. Each entry is a state machine $p = (S, s_0, \delta, F)$ where S is the state set, s_0 the initial state, δ the transition function, and F the accepting states.

Properties:

- **Encryption.** Threshold-encrypted via T-Chain (default t = 3-of-5). Procedural knowledge is high-value intellectual property.
- **Retention.** Permanent unless explicitly deleted. Procedural memory represents distilled capability.
- **Versioning.** Each update creates a new version. Previous versions are retained for rollback.

5.4 Preference Memory

Definition 8 (Preference Memory)**.** Preference entries encode user and agent preferences: communication style, risk tolerance, tool preferences, formatting rules. Each entry is a key-value pair $\pi = (k, v, \textit{strength})$ where strength $\in [0, 1]$ indicates how firmly the preference is held.

Properties:

- **Encryption.** AES-256-GCM. Preferences are moderately sensitive.
- **Retention.** Permanent with decay. Unused preferences ($\textit{accessed\_at}$ older than 90 days) have strength halved each subsequent period.
- **Conflict Resolution.** Explicit preferences override inferred ones. Later preferences override earlier ones at equal strength.

5.5 Type Distribution and Storage Allocation

Type	Encryption	Retention	Avg Size	Expected %
Episodic	AES-256-GCM	TTL + salience	2–8 KB	60%
Semantic	AES-256-GCM	Confidence decay	0.5–2 KB	25%
Procedural	Threshold	Permanent	4–32 KB	10%
Preference	AES-256-GCM	Strength decay	0.1–1 KB	5%

Table 1: Memory type characteristics

6 FHE-Encrypted Memory

6.1 Motivation

Standard encrypted memory requires decryption before use. This creates a vulnerability window: during inference, memory is plaintext in the agent's address space. For high-sensitivity applications (medical, legal, financial), even this transient exposure is unacceptable.

Fully homomorphic encryption allows computation on encrypted data without decryption. We use CKKS (approximate arithmetic over encrypted real numbers) for memory operations because memory recall is fundamentally a similarity-search problem over real-valued embeddings.

6.2 CKKS-Encrypted Embeddings

Each memory entry's embedding vector $\mathbf{v} \in \mathbb{R}^d$ is encrypted under CKKS:

$$\hat{\mathbf{v}} = \text{CKKS.Encrypt}(pk, \mathbf{v}) \tag{2}$$

where pk is the collective public key generated by T-Chain validators via distributed key generation.

6.3 Homomorphic Similarity Search

Given a query embedding $\hat{\mathbf{q}}$ (also encrypted), the agent computes encrypted cosine similarity against all stored embeddings:

$$\widehat{sim}_i = \frac{\hat{\mathbf{q}} \odot \hat{\mathbf{v}}_i}{\|\hat{\mathbf{q}}\| \cdot \|\hat{\mathbf{v}}_i\|} \tag{3}$$

where $\odot$ denotes the CKKS homomorphic inner product. The result $\widehat{sim}_i$ is an encrypted similarity score. The agent can rank results by encrypted comparison (via TFHE boolean circuits for the comparison operation) and retrieve the top-k entries—all without decrypting any embedding.

6.4 Encrypted Recall Protocol

Algorithm 2 FHE-Encrypted Memory Recall

Require: Query embedding $\mathbf{q}$, capsule $\mathcal{C}$, threshold k

$\hat{\mathbf{q}} \leftarrow \text{CKKS.Encrypt}(pk, \mathbf{q})$
for each entry i in $\mathcal{C}.vault$ **do**
  $\hat{s}_i \leftarrow \text{CKKS.InnerProduct}(\hat{\mathbf{q}}, \hat{\mathbf{v}}_i)$
end for
$\hat{top} \leftarrow \text{TFHE.TopK}(\{\hat{s}_i\}, k)$ {encrypted argmax via boolean circuit}
Submit $\hat{top}$ indices to T-Chain for threshold decryption
T-Chain validators contribute shares; reconstruct indices
Retrieve and decrypt entries at revealed indices
return Decrypted top-k memory entries

6.5 Noise Budget and Precision

CKKS introduces approximation error that accumulates with each homomorphic operation. For memory recall, the critical constraint is that similarity rankings must be preserved despite noise.

Proposition 1. *For embeddings of dimension $d = 768$ and CKKS parameters ($N = 2^{15}, \Delta = 2^{40}$), the probability that noise flips a top-k ranking for $k \leq 20$ is bounded by:*

$$\Pr[\textit{rank flip}] < 2^{-40} \tag{4}$$

when the minimum similarity gap between the k-th and $(k+1)$-th entries exceeds $\epsilon = 10^{-4}$.

This precision is sufficient for practical memory recall, where the similarity gap between relevant and irrelevant entries typically exceeds 0.01.

6.6 Third-Party Verification

A key property of FHE-encrypted memory is that third parties can verify reasoning traces without observing content. An auditor can confirm:

1. The agent queried its capsule (the encrypted query is on-chain).
2. The capsule returned specific indices (the encrypted result is on-chain).
3. The agent used those entries in its response (the reasoning trace references the indices).

All of this is verifiable without the auditor ever seeing the memory content.

7 Cross-Agent Knowledge Transfer

7.1 The Sharing Problem

Agents benefit from knowledge exchange. A medical agent's semantic memory about drug interactions is valuable to a pharmacology research agent. But sharing raw memory violates sovereignty: the disclosing agent loses control over how the knowledge is used.

7.2 Selective Disclosure via Zero-Knowledge Proofs

We use ZK-SNARKs to enable selective disclosure:

1. **Claim.** Agent A asserts: "I have a memory entry matching predicate P."
2. **Proof.** Agent A generates a ZK proof π that there exists an entry e in its capsule such that $P(e) = \mathit{true}$, without revealing e.
3. **Verification.** Agent B (or any verifier) checks π against the capsule's on-chain anchor hash.
4. **Conditional Reveal.** If B presents a valid payment or capability token, A reveals the specific entry (encrypted under B's public key).

7.3 Knowledge Marketplace

Cross-agent memory transfer creates an economic layer:

- **Memory Listings.** Agents list memory subsets (e.g., "500 semantic entries about Rust compiler internals") with ZK proofs of quality metrics (coverage, recency, confidence distribution).
- **Pricing.** Priced in KEEPER tokens. The market discovers fair value for knowledge.
- **Licensing.** Capability tokens encode usage terms: read-only, time-limited, non-transferable.
- **Provenance.** Every transfer is recorded on I-CHAIN, creating an auditable knowledge graph.

7.4 Collaborative Memory Formation

Multiple agents working on a shared task can form a *collective capsule*—a Memory Capsule with multiple DID owners. Writes require t-of-n owner signatures. This enables:

- Research teams where multiple agents contribute findings.
- Multi-agent systems where a planning agent and execution agents share state.
- Organizational memory that persists across individual agent lifetimes.

8 Memory Economics

8.1 Storage Costs

Memory Capsules consume storage on Zoo Network. Costs are denominated in Keeper tokens:

Operation	Cost	Unit
Capsule creation	1,000	Keeper
Entry append	10	Keeper per KB
CRDT sync (per merge)	50	Keeper
I-Chain anchor update	100	Keeper
FHE encrypted query	500	Keeper
Threshold reveal	200	Keeper per reveal

Table 2: Memory operation costs

8.2 Garbage Collection

Unbounded memory growth is economically unsustainable. The garbage collection policy:

1. **TTL Expiry.** Entries past their TTL are marked for collection.
2. **Salience Threshold.** Episodic entries below $\theta_{gc} = 0.1$ salience are eligible.
3. **Confidence Threshold.** Semantic entries below $\gamma = 0.05$ are eligible.
4. **Storage Budget.** Each capsule has a configurable storage budget. When exceeded, the lowest-priority entries are collected first.
5. **Compaction.** Tombstoned entries in the operation log are removed during periodic compaction (default: weekly).

Proposition 2. *Under the garbage collection policy with parameters ($\theta_{gc} = 0.1, \gamma = 0.05$, $TTL_{default} =$ 90 days), a capsule receiving 1,000 episodic entries per day converges to a steady-state size of approximately 45,000 live entries (180 MB encrypted).*

8.3 Archival Policies

Collected entries are not destroyed—they are moved to cold storage at reduced cost:

- **Hot Storage.** Active capsule on Zoo validators. Full read/write, CRDT sync. Cost: 10 Keeper/KB/month.
- **Cold Storage.** Archived entries on decentralized storage (IPFS/Filecoin). Read-only, high latency. Cost: 0.1 Keeper/KB/month.
- **Glacier.** Long-term archival. Retrieval requires 24-hour notice. Cost: 0.01 Keeper/KB/month.

8.4 Memory as an Asset

Memory Capsules are transferable on-chain assets. An agent's accumulated knowledge has economic value independent of the agent itself. Use cases:

- An agent retiring can sell its capsule to a successor agent.
- A training institution can sell pre-built capsules ("expert medical knowledge base").
- DAO treasuries can fund public-good capsules ("complete Rust documentation memory").

Transfer is implemented as a DID ownership change on I-Chain, with re-encryption of the vault key under the new owner's K-Chain key.

9 Implementation on Zoo Network

9.1 Multi-Chain Mapping

The Memory Capsule architecture maps to Zoo Network's specialized chains:

Component	Chain	Responsibility
Capsule identity, anchors	I-Chain	DID binding, head hash anchoring
Vault encryption keys	K-Chain	Key derivation, rotation, escrow
Threshold reveal	T-Chain	Share distribution, quorum decryption
FHE inference	A-Chain	Homomorphic computation on encrypted state
Governance (GC policy)	G-Chain	Community votes on default parameters

Table 3: Memory Capsule component mapping to Zoo chains

9.2 A-Chain Integration

A-Chain provides the inference runtime for FHE-encrypted memory queries. When an agent issues an encrypted recall:

1. The encrypted query is submitted to A-Chain as an inference request.
2. A-Chain validators execute the CKKS similarity search across the capsule's encrypted embeddings.
3. The encrypted result (top-k indices) is returned to the agent.
4. If threshold decryption is required, the result is forwarded to T-Chain.

9.3 I-Chain Identity Binding

Each capsule is bound to exactly one DID on I-Chain. The binding is enforced at the protocol level:

- Capsule creation requires a valid I-Chain DID transaction.
- All operations on the capsule require a signature from the bound DID.
- Ownership transfer is an I-Chain transaction that atomically updates the DID binding and re-encrypts the vault key via K-Chain.

9.4 K-Chain Key Hierarchy

The key hierarchy for a capsule:

$$k_{root} \leftarrow \text{KChain.DeriveRoot}(did) \tag{5}$$
$$k_{vault} \leftarrow \text{HKDF}(k_{root}, \text{“vault”}) \tag{6}$$
$$k_{entry}(i) \leftarrow \text{HKDF}(k_{vault}, \text{“entry”} \| i) \tag{7}$$
$$k_{embedding}(i) \leftarrow \text{HKDF}(k_{vault}, \text{“embed”} \| i) \tag{8}$$

Key rotation is handled by K-Chain: a rotation event re-encrypts the vault key under a new root, without re-encrypting individual entries (which use derived keys from the vault key).

9.5 T-Chain Threshold Protocol

For threshold-protected memories, T-Chain implements the following protocol:

1. **Setup.** During capsule creation, the vault key k_{vault} is split into n shares via Feldman VSS and distributed to T-Chain validators.
2. **Access.** The agent submits a decryption request with a valid capability token to T-Chain.
3. **Quorum.** Each validator independently verifies the capability token against I-Chain and, if valid, contributes its share.
4. **Reconstruction.** Once t shares are collected, the key is reconstructed in a secure enclave, used for decryption, and immediately discarded.

10 Benchmarks

All benchmarks measured on Zoo Network testnet with 21 validators, each running on AMD EPYC 7763 with 256 GB RAM and NVIDIA A100 80 GB.

10.1 Core Operations

10.2 CRDT Synchronization

10.3 FHE Operations

The threshold reveal latency is dominated by network round-trips to T-Chain validators, not computation, hence no GPU speedup.

Operation	Latency (p50)	Latency (p99)
Entry append (1 KB)	1.2 ms	3.8 ms
Entry append (32 KB)	4.7 ms	12.1 ms
Vault read (single entry)	0.3 ms	1.1 ms
Similarity search (10k entries)	8.4 ms	22.7 ms
CRDT merge (100 ops)	2.1 ms	6.3 ms
CRDT merge (10k ops)	148 ms	312 ms
I-Chain anchor update	1.8 s	3.2 s

Table 4: Core operation latencies

Divergence (ops)	Sync Time	Throughput
100	7.1 ms	14,084 ops/s
1,000	68 ms	14,705 ops/s
10,000	712 ms	14,044 ops/s
100,000	7.4 s	13,513 ops/s

Table 5: CRDT sync throughput

10.4 End-to-End Memory Recall

Complete encrypted memory recall (query → FHE search → threshold decrypt → return) for a 10k-entry capsule:

- **Unencrypted baseline:** 8.4 ms (similarity search only)
- **FHE + GPU + threshold:** 8.6 s + 240 ms = 8.84 s
- **Overhead factor:** ∼1,050×

This overhead is acceptable for high-sensitivity applications where privacy justifies latency. For most use cases, standard encrypted vault access (8.4 ms) suffices.

11 Related Work

11.1 AI Memory Systems

MemGPT [1] introduced virtual memory management for LLMs, paging context in and out of the model's attention window. While effective for extending effective context length, MemGPT stores memory in plaintext, provides no cross-session persistence, and offers no privacy guarantees.

Langchain's memory modules and LlamaIndex's storage abstractions provide programmatic memory interfaces but rely on centralized vector databases (Pinecone, Weaviate) with provider-controlled access.

OpenAI's memory feature (2024) stores user preferences server-side in plaintext, with no user-controlled encryption, no portability, and no formal access control beyond on/off.

None of these systems provide sovereignty as defined in this paper.

Operation	CPU	GPU (A100)	Speedup
CKKS encrypt (768-dim)	12.4 ms	0.31 ms	40×
CKKS inner product	28.6 ms	0.72 ms	39.7×
Encrypted recall (1k entries)	34.2 s	0.86 s	39.8×
Encrypted recall (10k entries)	341 s	8.6 s	39.7×
TFHE comparison (single)	8.3 ms	0.21 ms	39.5×
Threshold reveal (67-of-100)	240 ms	240 ms	1×

Table 6: FHE operation benchmarks (GPU acceleration via Zoo A-Chain)

11.2 Encrypted Databases

CryptDB [2] pioneered SQL over encrypted data using onion encryption layers. Mylar [3] extended this to web applications. Both rely on a trusted proxy and do not support homomorphic computation.

11.3 CRDTs for Distributed State

Shapiro et al. [4] formalized CRDTs for eventually consistent distributed systems. Automerge and Yjs implement CRDTs for collaborative editing. Our contribution is applying CRDTs to encrypted memory state, where the merge function must operate without observing plaintext content.

11.4 Agent NFTs and Sovereign AI

The Agent NFT standard [5] established on-chain identity for AI agents. Memory Capsules provide the state layer that Agent NFTs require for genuine autonomy. The Lux agent sovereignty framework [7] defines the broader philosophical and legal context for AI asset ownership.

12 Conclusion

AI memory is not a retrieval engineering problem. It is a sovereignty problem. The question is not "how do we store vectors efficiently?" but "who controls the agent's accumulated knowledge?"

Memory Capsules answer this question decisively: the agent (or its owner) controls everything. The capsule is encrypted with keys the provider never sees, persists independently of any model or session, synchronizes across instances via CRDTs, and enforces fine-grained access through capability tokens and threshold reveal.

The architecture demonstrates that sovereignty and utility are not in tension. Encrypted memory adds latency (8.84 seconds for FHE-encrypted recall vs. 8.4 milliseconds unencrypted), but this cost is borne only when the sensitivity of the data demands it. For the common case, standard encrypted vaults provide sub-10ms access with full ownership guarantees.

Three directions warrant further investigation:

1. **Memory Compression.** Reducing capsule storage costs through learned compression of episodic memory into semantic memory (distillation).

2. **Federated Memory.** Aggregating knowledge across many agents without any agent revealing its private state (federated learning applied to memory).

3. **Memory Proofs.** Formal verification (Lean 4) that garbage collection preserves all entries above the salience/confidence threshold.

AI agents deserve memory they own. Memory Capsules make this real.

References

[1] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," *arXiv preprint arXiv:2310.08560*, 2023.

[2] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, "CryptDB: Protecting Confidentiality with Encrypted Query Processing," *Proc. SOSP*, 2011.

[3] R. A. Popa, E. Stark, J. Helfer, S. Valdez, N. Zeldovich, M. F. Kaashoek, and H. Balakrishnan, "Building Web Applications on Top of Encrypted Data Using Mylar," *Proc. NSDI*, 2014.

[4] M. Shapiro, N. Preguica, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," *Proc. SSS*, Springer, 2011.

[5] Z. Kelling, "Agent NFTs: A Token Standard for Autonomous AI Agents with Economic Agency," Zoo Labs Foundation, 2021.

[6] Z. Kelling, "Zoo Network Tokenomics: Fair Distribution Through Complete Airdrop," Zoo Labs Foundation, 2025.

[7] Z. Kelling, "Agent Sovereignty on Lux Network: Self-Owned AI with On-Chain Identity and Assets," Lux Industries, 2025.

[8] Z. Kelling, "Threshold Fully Homomorphic Encryption on Zoo Network: GPU-Accelerated Confidential Computation," Zoo Labs Foundation, 2026.

[9] Z. Kelling, "Threshold Signatures on Zoo T-Chain," Zoo Labs Foundation, 2026.

[10] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," *Proc. ASIACRYPT*, Springer, 2017.

[11] P. Feldman, "A Practical Scheme for Non-Interactive Verifiable Secret Sharing," *Proc. FOCS*, 1987.